

Internal Project Visibility & Presence System: Architectural Blueprint & Implementation Strategy

1. Executive Summary

1.1 Strategic Intent and System Vision

In the landscape of 2025, the internal tooling ecosystem for IT consultancies faces a dichotomy: on one side, bloated enterprise resource planning (ERP) systems that enforce rigidity and surveillance; on the other, ephemeral spreadsheets and chat threads that lack persistence and structure. For a consultancy of 15 generalists, specifically one operating in a hybrid, project-based model, neither extreme is acceptable. The requirement is for a "Goldilocks" solution—a system that provides high-fidelity visibility into daily operations without the cultural corrosion associated with time-tracking or invasive monitoring.

This document outlines the architectural and implementation strategy for the **Internal Project Visibility & Presence System**. This platform is designed not merely as a status reporting tool, but as a low-friction institutional memory bank. It answers three fundamental questions: *Who is working? What is their primary focus? Are they blocked?* By strictly limiting the scope to these signals and excluding chat, time-tracking, and AI-driven analysis, the system optimizes for honesty and speed.

The architectural decision-making process detailed herein prioritizes "boring," implementation-grade technology that ensures immediate shippability while baking in the capability for future scaling. We reject the premature complexity of microservices in favor of a strictly typed, **Modular Monolith**. This approach leverages the stability of Node.js and PostgreSQL to deliver a robust backend, while modern frontend practices using React ensure a responsive user experience. Crucially, the system utilizes **Row-Level Security (RLS)** within the PostgreSQL data layer to enforce multi-tenancy at the kernel level. This strategic choice ensures that while the Minimum Viable Product (MVP) serves a single internal tenant, the underlying architecture is "SaaS-ready" from day one, capable of hosting thousands of distinct organizations without significant refactoring.

1.2 The "Boring Tech" Manifesto

The guiding philosophy for this implementation is "Innovation Tokens," a concept suggesting that a team has a limited capacity for adopting novel technologies before complexity overwhelms delivery. For this MVP, we spend zero innovation tokens on infrastructure. We rely on battle-tested technologies—TypeScript, React, NestJS, and PostgreSQL—to minimize the risk of "resume-driven development." The innovation lies not in the stack itself, but in the

rigorous application of domain boundaries within a monolithic codebase and the sophisticated use of database-native security features to solve the multi-tenancy problem elegantly.¹

This report serves as the definitive guide for the senior engineering team. It moves beyond generic recommendations to provide opinionated, hard constraints on how data flows, how modules communicate, and how security is enforced. It is a blueprint for a system that is shippable today, maintainable for the next five years, and scalable to a commercial product when the business dictates.

2. Architectural Principles & Topology

2.1 The Case for a Modular Monolith

In 2025, the industry has largely corrected course from the "microservices by default" mania of the previous decade. For a team of 15 users and a single engineering squad, microservices introduce catastrophic overhead: distributed tracing complexities, network latency, eventual consistency issues, and infrastructure costs that far outweigh their benefits.¹ The fallacy that microservices are required for modularity has been debunked; modularity is a function of code structure, not deployment topology.

We will adopt a **Modular Monolith** architecture. This places all business logic within a single deployable unit (a single Node.js process) but enforces strict boundaries between logical modules (e.g., Auth, Presence, Projects) via code structure and strict encapsulation.¹

The operational reality of a 15-person consultancy does not justify the complexity of orchestration layers like Kubernetes or service meshes like Istio. A modular monolith allows for atomic commits, simplified integration testing, and zero-latency in-memory communication between domains. Features that span multiple domains—such as checking if a user is blocked on a project they just joined—can be deployed instantly without the coordination overhead of versioning multiple independent services.⁴

However, "Monolith" does not imply "Spaghetti Code." We define strict architectural boundaries. The Presence module, for example, will not be permitted to import the User repository directly. Instead, it must utilize the public API of the `IamModule`. This discipline, enforced via linting rules (e.g., `dependency-cruiser`) and framework constraints (NestJS module encapsulation), preserves the option to "peel off" a module into a standalone microservice in the future if specific scaling dynamics demand it.³

2.2 System Context & Separation of Concerns

The system architecture follows a variant of **Clean Architecture** (often referred to as Onion

or Hexagonal Architecture), ensuring that business logic remains independent of external frameworks, UI concerns, and database drivers. This separation is critical for maintainability and testing.⁶

The system comprises four distinct layers:

1. **Presentation Layer (Frontend):** A React Single Page Application (SPA). Its responsibility is strictly limited to user interaction, state display, and optimistic UI updates. It contains *no* business rules regarding data validity, ensuring that the backend remains the single source of truth.
2. **Interface Layer (API):** REST Controllers that receive HTTP requests, validate inputs via Data Transfer Objects (DTOs), and delegate execution to the appropriate application services. This layer handles the translation between HTTP protocols (status codes, headers) and domain logic.
3. **Application Layer:** This layer contains the specific use cases of the system (e.g., "Submit Daily Check-in", "Archive Project"). It orchestrates the flow of data between domain entities and repositories but holds no persistent state itself.
4. **Domain Layer:** The heart of the software. This contains the entities (User, DailyStatus, Project) and the core business rules (e.g., "A user cannot check in twice on the same day"). This layer is pure TypeScript and has no dependencies on the database or the web framework.
5. **Infrastructure Layer:** The outermost layer containing implementations of repositories (PostgreSQL interactions via TypeORM or Prisma), email adapters, and logging mechanisms.

2.3 Technology Stack Selection

The stack selection prioritizes reliability, strict typing, and developer velocity. We avoid bleeding-edge runtimes or experimental frameworks to ensure stability for the internal tool.

Table 1: Technology Stack Selection & Justification

Component	Technology Selected	Justification
Backend Framework	NestJS (Node.js)	Unlike Express, NestJS provides an opinionated structure (Modules, Controllers, Services) that enforces the Modular Monolith pattern. Its dependency injection system is crucial for

		managing complex relationships between modules without tight coupling. ⁸
Frontend Library	React + Vite	React remains the industry standard for component-based UIs. ¹¹ Vite provides a superior developer experience with instant HMR (Hot Module Replacement) compared to Webpack. We explicitly avoid Next.js for the frontend to maintain a clean separation between the Client and API, simplifying future mobile app development. ¹²
Database	PostgreSQL (v15+)	chosen for its robust relational data integrity, JSONB support for flexible schemas, and most critically, Row-Level Security (RLS) , which underpins our multi-tenant security model. ¹³
Language	TypeScript	Used across the entire stack (Backend and Frontend). Shared DTOs ensure end-to-end type safety, eliminating an entire class of runtime errors and reducing the need for extensive contract testing. ¹⁵
State Management	TanStack Query	Manages server state (caching, polling, optimistic updates) on the frontend.

		This is preferred over Redux as the application is primarily a view of remote server data. ¹⁶
--	--	--

3. Backend Architecture: The Modular Monolith Implementation

3.1 Framework: NestJS vs. Express/Fastify

While frameworks like Fastify offer raw performance advantages in synthetic benchmarks¹⁸, **NestJS** is the selected framework for this architecture. The primary bottleneck for an internal tool (and even a moderately scaled SaaS) is rarely raw I/O throughput, but rather developer organization and maintainability.

Express.js, while ubiquitous, is unopinionated to a fault. In a team environment, this often leads to "Module Spaghetti," where business logic leaks into controllers and database queries are scattered throughout the codebase. NestJS leverages TypeScript decorators and a dedicated Dependency Injection (DI) container to force developers to think in terms of architectural boundaries. It creates a standardized way to define Modules, which aligns perfectly with our Modular Monolith strategy.⁹

Furthermore, NestJS's ecosystem includes built-in support for guards, interceptors, and pipes, which allows us to implement cross-cutting concerns like validation and authentication consistently across all modules without code duplication.¹

3.2 Module Structure and Boundaries

The backend codebase will be organized by **Domain Features** rather than technical layers. We reject the traditional structure of grouping all controllers in one folder and all services in another. Instead, a top-level `src/modules` directory will contain self-contained directories for each business capability.¹

Detailed Directory Structure:

```
src/
├── app.module.ts # Root module orchestrating imports
├── main.ts # Application entry point (Bootstrap)
├── common/ # Shared utilities (Strictly Infrastructure)
│   ├── decorators/ # Custom decorators (e.g., @CurrentUser)
│   ├── filters/ # Global Exception filters
│   └── guards/ # Auth and RLS guards
```

```

|   └─ interceptors/ # Response transformation and logging
|   └─ modules/
|   └─ auth/ # Authentication Module (JWT handling)
|       └─ auth.controller.ts
|       └─ auth.service.ts
|       └─ strategies/ # Passport strategies
|   └─ iam/ # Identity & Access Management (Users, Tenants)
|       └─ iam.module.ts
|       └─ dto/ # Data Transfer Objects
|       └─ domain/ # User Entities
|       └─ infrastructure/ # User Repository implementation
|   └─ presence/ # Core Domain: Daily Check-ins & Status
|       └─ presence.module.ts
|       └─ dto/ # create-checkin.dto.ts
|       └─ presence.service.ts
|   └─ projects/ # Project Management Domain
|       └─ projects.controller.ts
|       └─ projects.service.ts

```

Architectural Rule of Thumb:

- **Strict Encapsulation:** The Presence module cannot import the User repository directly. Accessing user data must be done by injecting the exported `IamService`. This prevents a scenario where changes to the User database schema break unrelated modules.⁶
- **The Shared Kernel:** The common folder contains *only* infrastructure concerns (e.g., logging, exception filters, generic RLS middleware). It does *not* contain shared business logic, preventing the creation of a "god object" or utility folder that becomes a dumping ground for miscellaneous code.¹

3.3 Communication Between Modules

In a microservices architecture, modules communicate via network calls (REST/gRPC), introducing latency and failure modes. In our modular monolith, modules communicate via **in-process method calls**, which are reliable and fast. However, to maintain loose coupling, we enforce specific communication patterns:

1. **Public API:** Each module (e.g., `IamModule`) exports a specific set of services (`IamService`) meant for external consumption. Internal helper services or repositories are not exported from the module, making them private by default.
2. **Synchronous vs. Asynchronous:**
 - **Synchronous:** For operations that require immediate consistency (e.g., checking if a Project exists before creating a Check-in), we use direct service injection.
 - **Asynchronous (Phase 2):** For side effects (e.g., "When a user checks in, update the daily analytics cache"), we will utilize NestJS's internal `EventEmitter`.⁸ This creates a temporal decoupling, ensuring that a failure in the analytics calculation does not

block the user's check-in response.

4. Database Design & Data Governance

4.1 Database Engine: PostgreSQL

PostgreSQL is the mandatory database choice. While NoSQL databases like MongoDB offer schema flexibility, the data model for a presence system is highly relational (Users belong to Projects; Check-ins belong to Users). We require the referential integrity enforced by foreign keys to ensure data quality—a critical component of the "Honesty" optimization goal.²¹

Furthermore, PostgreSQL's advanced features, specifically JSONB columns, allow us to handle semi-structured data (like "Blocker Reasons" or future metadata extensions) without abandoning the rigor of a relational schema. Most importantly, PostgreSQL's implementation of **Row-Level Security (RLS)** is superior to competitors, providing the foundation for our security architecture.¹³

4.2 Entity Relationship Diagram (ERD) and Schema Definitions

The schema is designed to support multi-tenancy natively, even though the MVP is internal-only. Every primary table includes a `tenant_id` discriminator.

1. Tenants Table (Foundation for SaaS)

This table isolates data boundaries. For the MVP, a single row (the "Default Tenant") is created during migration.

- `id`: UUID (Primary Key)
- `name`: VARCHAR(255)
- `created_at`: TIMESTAMPTZ
- `status`: ENUM ('active', 'suspended')

2. Users Table

- `id`: UUID (Primary Key)
- `tenant_id`: UUID (Foreign Key -> Tenants.id) [Indexed]
- `email`: VARCHAR (Unique Constraint scoped to `tenant_id`)
- `password_hash`: VARCHAR
- `full_name`: VARCHAR
- `role`: ENUM ('admin', 'member')
- `is_active`: BOOLEAN

3. Projects Table

- `id`: UUID (Primary Key)
- `tenant_id`: UUID (Foreign Key -> Tenants.id) [Indexed]

- name: VARCHAR
- description: TEXT
- owner_id: UUID (Foreign Key -> Users.id)
- status: ENUM ('active', 'archived', 'on_hold')
- created_at: TIMESTAMPTZ

4. DailyCheckIns Table (Core Transactional Table)

This table captures the daily pulse of the organization.

- id: UUID (Primary Key)
- tenant_id: UUID (Foreign Key -> Tenants.id) [Indexed]
- user_id: UUID (Foreign Key -> Users.id)
- project_id: UUID (Foreign Key -> Projects.id)
- date: DATE (Indexed, defaults to CURRENT_DATE)
- status: ENUM ('in_office', 'remote', 'on_leave')
- intent: VARCHAR(120) (Strict limit enforced)
- is_blocked: BOOLEAN
- block_reason_category: ENUM ('client', 'teammate', 'tech', 'other')
- block_reason_text: TEXT
- created_at: TIMESTAMPTZ
- *Constraint*: UNIQUE (user_id, date) - Prevents duplicate check-ins.

5. DailyOutcomes Table

- id: UUID (Primary Key)
- tenant_id: UUID (Foreign Key -> Tenants.id) [Indexed]
- check_in_id: UUID (Foreign Key -> DailyCheckIns.id)
- outcome_text: TEXT
- created_at: TIMESTAMPTZ

4.3 Indexing Strategy for RLS and Performance

Indexing is critical not just for query speed, but for RLS efficiency. Since RLS policies effectively append a WHERE tenant_id = '...' clause to every query, a lack of indexes on the tenant_id column would force the database to perform full table scans for every operation, destroying performance as the dataset grows.²²

Strategic Indexes:

1. **Tenant Isolation Index:** Every table **MUST** have a B-Tree index on tenant_id.
 - CREATE INDEX idx_checkins_tenant ON daily_check_ins (tenant_id);
2. **Dashboard Compound Index:** The dashboard frequently queries "All check-ins for Tenant X on Date Y." A compound index covers this exact query pattern.
 - CREATE INDEX idx_checkins_tenant_date ON daily_check_ins (tenant_id, date DESC);
3. **Foreign Key Indexes:** All foreign keys (user_id, project_id) must be indexed to speed up JOIN operations, which are common when assembling the dashboard view.

5. Multi-Tenancy Strategy: The Row-Level Security (RLS) Approach

5.1 The Shared Database/Shared Schema Model

For the MVP and up to hundreds of tenants (SaaS Phase), we will use a **Shared Database, Shared Schema** approach with a discriminator column (`tenant_id`). This is the most operationally efficient model. It simplifies database migrations (run once for all tenants) and reduces infrastructure costs compared to a "Database-per-Tenant" model, which involves managing hundreds of distinct connection pools.²¹

The primary risk of the Shared Schema model is **Data Leakage**. If a developer forgets to add `WHERE tenant_id = ?` to a query, one client could see another client's data. To mitigate this catastrophic risk, we will not rely on application-level filtering. Instead, we use **PostgreSQL Row-Level Security (RLS)**.¹³

5.2 RLS Implementation Details

RLS moves the isolation logic from the application code (Node.js) to the database kernel (PostgreSQL). Even if the application code attempts to `SELECT * FROM daily_check_ins`, the database will only return rows permitted by the active policy.

The Mechanism:

1. **Database Policy:** We define a policy on every table that restricts access based on a session variable.
2. **Session Context:** When an HTTP request arrives, the backend validates the JWT, extracts the `tenantId`, and sets a session-local variable in the Postgres connection (e.g., `app.current_tenant`).

SQL Policy Implementation:

SQL

```
-- 1. Enable RLS on the table
```

```
ALTER TABLE daily_check_ins ENABLE ROW LEVEL SECURITY;
```

```
-- 2. Create the Isolation Policy
```

```
-- "current_setting" retrieves the session variable set by the API
```

```
CREATE POLICY tenant_isolation_policy ON daily_check_ins
  USING (tenant_id = current_setting('app.current_tenant')::uuid);
```

```
-- 3. Force RLS for Table Owners
```

```
-- This ensures that even the db user used by the app cannot bypass RLS
```

```
ALTER TABLE daily_check_ins FORCE ROW LEVEL SECURITY;
```

Application Middleware (NestJS Interceptor):

We must implement a specific interceptor that wraps every database interaction in a transaction context where the tenant variable is set.

TypeScript

```
// Conceptual NestJS Interceptor for RLS
```

```
@Injectable()
```

```
export class RlsInterceptor implements NestInterceptor {
```

```
  constructor(private dataSource: DataSource) {}
```

```
  async intercept(context: ExecutionContext, next: CallHandler): Promise<Observable<any>> {
```

```
    const request = context.switchToHttp().getRequest();
```

```
    const tenantId = request.user?.tenantId; // Extracted from JWT
```

```
    if (!tenantId) {
```

```
      throw new InternalServerErrorException('Tenant ID missing from context');
```

```
    }
```

```
    // Wrap the request handling in a transaction
```

```
    return from(this.dataSource.transaction(async (manager) => {
```

```
      // Set the tenant context for this specific transaction
```

```
      await manager.query(`SET LOCAL app.current_tenant = '${tenantId}'`);
```

```
      // Proceed with the request handler
```

```
      const result = await lastValueFrom(next.handle());
```

```
      return result;
```

```
    }));
```

```
  }
```

```
}
```

Justification for RLS:

- **Defense in Depth:** It provides a safety net against developer error. Data isolation is

enforced by the database engine, not just the ORM.²³

- **Zero-Cost SaaS Conversion:** Moving from internal-only to SaaS requires zero schema changes. We simply create new Tenant records and issue JWTs with different IDs. The application logic remains identical.²⁵
- **Simplicity:** The application services remain clean. We do not need to pass tenantId into every service method or repository call; the context is handled implicitly by the infrastructure layer.

6. API Design & Integration Patterns

6.1 API Principles

We will adopt a **REST Level 2** maturity model. While GraphQL offers flexibility, it introduces complexity (schema stitching, N+1 query problems) that is unnecessary for this strict, small-scope data model. REST provides predictable performance and easy cacheability.

- **Resources:** URIs will represent resources (/projects, /check-ins).
- **Verbs:** Standard HTTP verbs (GET, POST, PATCH, DELETE) will be used semantically.
- **Statelessness:** No server-side sessions will be maintained; all state is encapsulated in the JWT.

6.2 Core Endpoints (MVP Scope)

Table 2: Key API Endpoints

Method	Endpoint	Description	Request Body (DTO)
Auth			
POST	/auth/login	Authenticate user via email/password. Returns HttpOnly cookie.	{email, password}
POST	/auth/logout	Clear the HttpOnly cookie.	-
Check-In			

POST	/check-ins	Submit daily status. Creates a new check-in record.	{status, projectId, intent, isBlocked,...}
GET	/check-ins/today	Check if the current user has already checked in today.	-
PATCH	/check-ins/:id/outcome	Update a check-in with the EOD outcome.	{outcomeText}
Dashboard			
GET	/dashboard/presence	Get aggregated presence summary and project lists.	-
GET	/dashboard/blockers	Get a dedicated list of active blockers.	-
Projects			
GET	/projects	List all active projects for the tenant.	-
POST	/projects	Create new project (Admin only).	{name, ownerId, description}

6.3 Security & Rate Limiting

- **Rate Limiting:** Implemented via npm @nestjs/throttler. We will set a conservative limit of 100 requests/minute per IP. While this is an internal tool, rate limiting protects against accidental loops in scripts or compromised user machines.
- **Input Validation:** We rely on class-validator and class-transformer within NestJS. All incoming payloads are validated against DTO classes. Invalid requests are rejected with 400 Bad Request before they ever reach the business logic, preventing "garbage in, garbage out."
- **CORS:** Cross-Origin Resource Sharing will be strictly configured to allow only the specific

frontend domain. Wildcard domains are prohibited.

7. Frontend Architecture & User Experience

7.1 Tech Stack: React + Vite + TanStack Query

The frontend is a standard Single Page Application (SPA). We utilize **Vite** for the build tool due to its extreme speed compared to Webpack, which significantly improves developer feedback loops.¹¹

State Management Philosophy:

State management is often the source of greatest complexity in frontend apps. We distinguish between two types of state:

1. **Server State (95% of data):** This is data that lives on the server (e.g., list of projects, current user's check-in). We will use **TanStack Query (React Query)** to manage this. It handles caching, background polling (crucial for near real-time dashboards), and optimistic updates out of the box. This is superior to Redux for this use case because the app is primarily a *view* of the server state.¹⁶
2. **Client State (UI):** This is ephemeral interface state (e.g., is the sidebar open? is the modal visible?). We will use **Zustand** for this due to its minimal boilerplate compared to Redux or Context API.

7.2 Directory Structure

```
src/
├── components/ # Shared UI components (Buttons, Cards)
│   └── ui/ # Shadcn/UI primitives (Accessible, unstyled components)
├── features/ # Feature-based folders (Colocation of concerns)
│   ├── check-in/ # Daily Check-in Logic
│   │   ├── components/ # Check-in Form, Status Selector
│   │   ├── hooks/ # useSubmitCheckin (React Query mutation)
│   │   └── api/ # checkinService.ts (Axios calls)
│   ├── dashboard/ # Dashboard Views
│   └── auth/ # Login screens
├── hooks/ # Global hooks (useTheme, useAuth)
└── lib/ # Axios instance, Utils
```

7.3 Optimistic UI Patterns for "Low Friction"

To meet the "Extremely low friction" requirement, the interface must feel instant. Users should never stare at a loading spinner after clicking "Submit."

- **Optimistic Check-in:** When a user clicks "Submit" on their daily check-in, the UI immediately transitions to the "Success" state and updates the local dashboard cache

while the API call happens in the background. If the API call fails (rare), the UI reverts and displays a toast notification.

- **Blocker Resolution:** Resolving a blocker on the dashboard immediately removes it from the list visually, reinforcing the action.

7.4 The "Anti-Surveillance" UI Design

The design must actively discourage micromanagement and reinforce trust. The UI is the primary interface through which the company culture is experienced.

- **No Timestamps:** The dashboard shows *who* checked in, but explicitly hides the exact *time* of check-in. This prevents "they were 5 minutes late" scrutiny and focuses on the *content* of the update.
- **Focus on Blockers:** The "Blockers Panel" will be the most prominent visual element (using Red/Orange accents), drawing attention to *problems the team needs help with*, rather than judging individual output.
- **No "History" Browsing:** The MVP allows users to see their own history (for memory), but managers cannot browse a "productivity log" of other users. The main view focuses entirely on "Today."

8. Security & Authentication Framework

8.1 Authentication Flow: HttpOnly Cookies

We will use **JSON Web Tokens (JWT)** for stateless authentication. However, we strictly reject storing JWTs in `localStorage`, which is vulnerable to Cross-Site Scripting (XSS) attacks. If an attacker can inject a script, they can read `localStorage` and steal the token.²⁶

The Secure Flow:

1. **Login:** User sends credentials (email, password) to the backend.
2. **Validation:** Backend validates credentials against the `password_hash` (using `bcrypt` or `argon2`).
3. **Token Issuance:** Backend signs a JWT containing the `userId` and `tenantId`.
4. **Cookie Attachment:** The backend sends the JWT in a `Set-Cookie` header with the following flags:
 - `HttpOnly`: The cookie cannot be accessed by client-side JavaScript (mitigates XSS).
 - `Secure`: The cookie is only sent over HTTPS.
 - `SameSite=Strict`: The cookie is only sent for first-party requests (mitigates CSRF).
5. **Transport:** The browser automatically attaches this cookie to all subsequent requests to the API.
6. **Refresh Strategy:** For the MVP, we use a long-lived Access Token (7 days) to minimize friction. In Phase 3, we will introduce a split Access/Refresh token rotation strategy for

enhanced security.

8.2 Security Headers

The backend will use the Helmet middleware to set standard security headers that protect the application from common web vulnerabilities:

- **Content-Security-Policy (CSP):** Restrict sources of scripts, images, and styles to authorized domains only.
- **X-Frame-Options: DENY** (Prevents the app from being embedded in iframes, stopping clickjacking attacks).
- **X-Content-Type-Options: nosniff** (Prevents the browser from MIME-sniffing a response away from the declared content-type).

9. Infrastructure & Deployment Strategy

9.1 Hosting: PaaS (Render or Railway)

For an internal tool with limited DevOps resources, managing Kubernetes clusters or raw EC2 instances is a poor use of engineering time. We will use a **Platform as a Service (PaaS)**.

Render.com or **Railway.app** are ideal choices for 2025.²⁹

Selected Topology (Render.com):

- **Web Service:** Dockerized Node.js API (NestJS). Render automatically detects the Dockerfile and builds it.
- **Static Site:** React Frontend. Served via Render's global CDN for low latency.
- **Database:** Managed PostgreSQL instance provided by Render. This includes automated daily backups and point-in-time recovery, which is essential for data safety.

9.2 CI/CD Pipeline

We will use **GitHub Actions** to automate the delivery pipeline, ensuring that "Shipping Fast" does not mean "Breaking Things."

Pipeline Steps (On Push to Main):

1. **Lint & Type Check:** Run eslint and tsc --noEmit. If this fails, the build stops.
2. **Build Docker Image:** Build the container to ensure the Dockerfile is valid.
3. **Integration Tests:** Spin up a temporary PostgreSQL service container and run the NestJS integration tests. This verifies that the RLS policies and API endpoints are functioning correctly.
4. **Deploy to Staging:** Automatically deploy the successful build to the staging environment on Render.
5. **Production Release:** Production deployment is triggered manually via a "Release" tag in

GitHub, providing a human gate for final verification.

10. Development Roadmap & Phasing

Phase 1: The "Honesty" MVP (Weeks 1-3)

- **Goal:** Replace the morning standup with data. Ship the absolute minimum to provide value.
- **Scope:**
 - Auth (Email/Pass) with HttpOnly Cookies.
 - Database Schema with basic RLS policies enabled.
 - Check-in Form (Status, Project, Intent, Blockers).
 - Company Dashboard (Presence list, Blockers list).
 - Basic Project creation (Admin only).
- **Explicitly Excluded:** User profile editing, Password reset (users must ask Admin), Historical reporting for managers, Mobile app, Slack integration.

Phase 2: Collaboration & Insight (Weeks 4-8)

- **Goal:** Facilitate help and remove blockers.
- **Features:**
 - "I can help" button on blockers (Notifies the blocked user).
 - **Slack Integration:** A cron job that posts a summary of active blockers to the #engineering channel at 10 AM daily. This moves the visibility to where the team already hangs out.
 - Personal History view (Last 30 days) so users can remember what they did for their own standups or reviews.

Phase 3: Scale & Intelligence (Month 3+)

- **Goal:** Institutional Memory and SaaS Prep.
 - **Features:**
 - Searchable history (e.g., "What did we do on Project X last month?").
 - Analytics (Blocker frequency by project—identifying "problem projects").
 - SaaS preparation (Stripe integration, Tenant creation flow).
-

11. Scalability & Future-Proofing

11.1 Scaling Users (15 -> 100+)

The architecture is designed to handle this growth without modification.

- **Database:** The chosen RLS architecture scales linearly. PostgreSQL can handle millions

of rows. At 100 users generating ~100 check-ins/day, we will generate only ~36,500 rows per year. We are nowhere near the limits of even the smallest database instance.

- **App Server:** Node.js is single-threaded. If CPU becomes a bottleneck (unlikely with this workload), the stateless architecture allows us to horizontally scale (spin up more containers) behind a load balancer immediately.
- **Frontend:** React Query handles caching aggressively. As the user count grows, the load on the API is minimized because clients serve stale data from the cache while revalidating in the background.

11.2 Transitioning to SaaS

The architecture is "SaaS-ready" from day one due to the `tenant_id` column and RLS.

- **Conversion Path:**
 1. Create a public "Sign Up" page that creates a new Tenant record.
 2. Update the Auth service to issue JWTs associated with the new tenant.
 3. **No code changes** are required in the core application logic (Projects/Presence modules) because they rely entirely on the RLS abstraction. The system already thinks it is multi-tenant; we just haven't added the second tenant yet.

12. Risk Management & Trade-offs

Table 3: Risk Matrix and Mitigation

Risk Category	Risk Description	Mitigation Strategy
Technical	RLS Complexity: Developers might find debugging RLS policies difficult, leading to "missing data" bugs where data exists but is hidden by the policy.	Implement comprehensive integration tests that explicitly test cross-tenant access. Create a "sudo" mode for admins only used for debugging that bypasses RLS (carefully guarded).
Technical	Monolith Coupling: Over time, modules might become tightly coupled (spaghetti code), negating the benefits of the architecture.	Enforce dependency-cruiser rules in the CI/CD pipeline to fail builds if modules import from each other's private directories or bypass the

		public API.
Cultural	Surveillance Fear: Users may feel "watched" or pressured to write "good" intents, leading to performative work rather than honest updates.	Leadership must explicitly frame this as a "Blocker Removal Tool," not a "Time Tracker." Disable timestamp display on the dashboard. Use the data to <i>help</i> , not to <i>blame</i> .
Adoption	Friction: If the check-in takes >30 seconds, devs will stop doing it.	Optimize the UI for speed. Pre-fill "Yesterday's Project" as the default selection. Support keyboard shortcuts (Cmd+Enter to submit).
Operational	Single Point of Failure: Shared DB for all tenants (in SaaS future). If the DB goes down, all customers go down.	Automated daily backups. Point-in-time recovery enabled. In Phase 4 (Massive Scale), we can migrate large tenants to dedicated database shards if necessary.

13. Conclusion

This architecture resists the urge to over-engineer. It uses a **Modular Monolith** to provide the structure needed for a professional engineering team without the operational overhead of microservices. It leverages **PostgreSQL RLS** to provide military-grade data isolation and a free ticket to a future SaaS business model.

By strictly limiting the scope to "Presence and Blockers" and removing surveillance signals (time tracking, AI monitoring), the system respects the "Generalist/Hybrid" culture of the consultancy. It is designed not just to be built, but to be lived in—fast, reliable, and invisible until it is needed.

Recommendation: Proceed immediately with the NestJS/React/Postgres stack. Prioritize the RLS implementation in the first sprint to secure the multi-tenant foundation.

Detailed Appendix: Implementation Specifications

A1. Database Schema (SQL Definition)

SQL

```
-- Enable UUID extension for primary keys
CREATE EXTENSION IF NOT EXISTS "uuid-ossf";

-- 1. Tenants Table
CREATE TABLE tenants (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name VARCHAR(255) NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 2. Users Table
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  email VARCHAR(255) NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  full_name VARCHAR(255) NOT NULL,
  is_admin BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  -- Ensure emails are unique within a tenant, but allow duplicates across tenants
  CONSTRAINT users_tenant_email_key UNIQUE (tenant_id, email)
);

-- 3. Projects Table
CREATE TABLE projects (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  name VARCHAR(255) NOT NULL,
  owner_id UUID REFERENCES users(id),
```

```

status VARCHAR(50) DEFAULT 'active', -- active, archived
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 4. Daily Check-ins Table
CREATE TABLE daily_check_ins (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  user_id UUID NOT NULL REFERENCES users(id),
  project_id UUID REFERENCES projects(id),
  date DATE NOT NULL DEFAULT CURRENT_DATE,
  status VARCHAR(50) NOT NULL, -- in_office, remote, on_leave
  intent VARCHAR(120), -- Strict limit for brevity
  is_blocked BOOLEAN DEFAULT FALSE,
  block_reason TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  -- One check-in per user per day
  CONSTRAINT unique_user_date UNIQUE (user_id, date)
);

-- 5. Daily Outcomes
CREATE TABLE daily_outcomes (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  user_id UUID NOT NULL REFERENCES users(id),
  date DATE NOT NULL DEFAULT CURRENT_DATE,
  outcome_text TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- INDEXES for Performance and RLS
CREATE INDEX idx_users_tenant ON users(tenant_id);
CREATE INDEX idx_projects_tenant ON projects(tenant_id);
CREATE INDEX idx_checkins_tenant ON daily_check_ins(tenant_id);
CREATE INDEX idx_checkins_date ON daily_check_ins(date);
CREATE INDEX idx_outcomes_tenant ON daily_outcomes(tenant_id);

```

A2. Row Level Security Policies

SQL

```

-- 1. Enable RLS on all sensitive tables
ALTER TABLE projects ENABLE ROW LEVEL SECURITY;
ALTER TABLE daily_check_ins ENABLE ROW LEVEL SECURITY;
ALTER TABLE daily_outcomes ENABLE ROW LEVEL SECURITY;
ALTER TABLE users ENABLE ROW LEVEL SECURITY;

-- 2. Define Policies
-- Projects: Users can see all projects in their tenant
CREATE POLICY tenant_isolation_projects ON projects
  USING (tenant_id = current_setting('app.current_tenant')::uuid);

-- Check-ins: Users can see all check-ins in their tenant (Visibility)
CREATE POLICY tenant_isolation_checkins ON daily_check_ins
  USING (tenant_id = current_setting('app.current_tenant')::uuid);

-- Check-ins INSERT: Users can only insert for their own user_id and tenant
-- This prevents a user from spoofing a check-in for someone else
CREATE POLICY insert_own_checkin ON daily_check_ins
  WITH CHECK (
    tenant_id = current_setting('app.current_tenant')::uuid
  AND
    user_id = (SELECT id FROM users WHERE email = current_setting('app.current_user_email'))
  );

-- 3. Force RLS
-- This is a critical security step. It ensures RLS applies even to the table owner.
ALTER TABLE projects FORCE ROW LEVEL SECURITY;
ALTER TABLE daily_check_ins FORCE ROW LEVEL SECURITY;

```

A3. API Interface Definitions (TypeScript DTOs)

1. Submit Check-in DTO

TypeScript

```

import { IsUUID, IsEnum, IsString, MaxLength, IsBoolean, IsOptional } from 'class-validator';

export enum WorkStatus {
  IN_OFFICE = 'In Office',

```

```

    REMOTE = 'Remote',
    ON_LEAVE = 'On Leave'
  }

  export class CreateCheckInDto {
    @IsEnum(WorkStatus)
    status: WorkStatus;

    @IsUUID()
    projectId: string;

    @IsString()
    @MaxLength(120, { message: 'Intent must be 120 chars or less' })
    intent: string;

    @IsBoolean()
    isBlocked: boolean;

    @IsOptional()
    @IsString()
    blockReason?: string;
  }

```

2. Project Dashboard Response Interface

TypeScript

```

export interface DashboardProjectCard {
  id: string;
  name: string;
  ownerName: string;
  activeMemberCount: number; // Count of check-ins today for this project
  hasBlockers: boolean; // True if any member on this project is blocked
}

export interface PresenceSummary {
  inOffice: number;
  remote: number;
  onLeave: number;
  blocked: number;
}

```

}

Works cited

1. Nest.js and Modular Architecture: Principles and Best Practices - Level Up Coding, accessed on January 17, 2026, <https://levelup.gitconnected.com/nest-js-and-modular-architecture-principles-and-best-practices-806c2cb008d5>
2. Monolithic vs Microservices Architecture: Pros and Cons for 2025 - Scalo, accessed on January 17, 2026, <https://www.scalosoft.com/blog/monolithic-vs-microservices-architecture-pros-and-cons-for-2025/>
3. Best Practices for Designing a Modular Monolithic Architecture - MultiQoS, accessed on January 17, 2026, <https://multiqos.com/blogs/modular-monolithic-architecture/>
4. Monolith vs Microservices: A Practical Guide with Examples in Node.js using TypeScript, accessed on January 17, 2026, <https://levelup.gitconnected.com/monolith-vs-microservices-a-practical-guide-with-examples-in-node-js-using-typescript-546c05eb3af6>
5. Beyond Microservices: The Emerging Post-Monolith Architecture for 2025 - DZone, accessed on January 17, 2026, <https://dzone.com/articles/post-monolith-architecture-2025>
6. Building a Scalable Express API Using Clean Architecture and a Modular Monolith (with TypeScript) | by Antoine TSINGA | Nov, 2025 | Medium, accessed on January 17, 2026, <https://medium.com/@mwwtstq/building-a-scalable-express-api-using-clean-architecture-and-a-modular-monolith-with-typescript-c855614b05dc>
7. deadislove/nestJS-modular-monolith-cqrs-event-sourcing-architecture-template - GitHub, accessed on January 17, 2026, <https://github.com/deadislove/nestJS-modular-monolith-cqrs-event-sourcing-architecture-template>
8. Microservices | NestJS - A progressive Node.js framework, accessed on January 17, 2026, <https://docs.nestjs.com/microservices/basics>
9. How to Structure Your NestJS App Without Losing Your Mind | by Luis Sarmiento - Medium, accessed on January 17, 2026, <https://medium.com/@lsarmientom010/how-to-structure-your-nestjs-app-without-losing-your-mind-02f553d05920>
10. Which JavaScript backend framework or runtime is best for 2025? #177376 - GitHub, accessed on January 17, 2026, <https://github.com/orgs/community/discussions/177376>
11. Top 16 React JS Development Services (2025 Founder's Guide) - Build Your App in 4-8 weeks | Bricks Tech, accessed on January 17, 2026, <https://www.brickstech.io/post/react-js-development-services-guide-for-founders>
12. Splitting Frontends + JWT Authentication with Cookies: Best Practice? : r/node -

- Reddit, accessed on January 17, 2026,
https://www.reddit.com/r/node/comments/1j1v4z2/splitting_frontends_jwt_authentication_with/
13. Mastering PostgreSQL Row-Level Security (RLS) for Rock-Solid Multi-Tenancy, accessed on January 17, 2026,
<https://ricofritzsche.me/mastering-postgresql-row-level-security-rls-for-rock-solid-multi-tenancy/>
 14. Multi-Tenancy using Row Level Security in Postgres | by Manish Chaulagain | Medium, accessed on January 17, 2026,
<https://medium.com/@manishchaulagain/multi-tenancy-using-row-level-security-in-postgres-2ebfd6871539>
 15. Node.js with TypeScript: Should You Make the Switch in 2025? - Abbacus Technologies, accessed on January 17, 2026,
<https://www.abbacustechologies.com/node-js-with-typescript-should-you-make-the-switch-in-2025/>
 16. React Query vs SWR: Which One is Better? - NamasteDev Blogs, accessed on January 17, 2026,
<https://namastedev.com/blog/react-query-vs-swr-which-one-is-better-4/>
 17. React Query or SWR: Which is best in 2025? - DEV Community, accessed on January 17, 2026,
<https://dev.to/rigalpatel001/react-query-or-swr-which-is-best-in-2025-2oa3>
 18. Express.js vs Fastify: An In-Depth Framework Comparison | Better Stack Community, accessed on January 17, 2026,
<https://betterstack.com/community/guides/scaling-nodejs/fastify-express/>
 19. Express or Fastify in 2025: What's the Right Node.js Framework for You? - Medium, accessed on January 17, 2026,
<https://medium.com/codetodeploy/express-or-fastify-in-2025-whats-the-right-node-js-framework-for-you-6ea247141a86>
 20. Recommended Folder Structure for Node(TS) 2025 - DEV Community, accessed on January 17, 2026,
https://dev.to/pramod_boda/recommended-folder-structure-for-nodets-2025-39jl
 21. Strict Data Isolation in Multi-tenant Systems with PostgreSQL | by Moyosoreoluwa Odusola, accessed on January 17, 2026,
<https://medium.com/@moyo.sore.oluwa/strict-data-isolation-in-multitenant-systems-with-postgresql-aa615052fe80>
 22. RLS Performance and Best Practices · supabase · Discussion #14576 - GitHub, accessed on January 17, 2026,
<https://github.com/orgs/supabase/discussions/14576>
 23. Underrated Postgres: Build Multi-Tenancy with Row-Level Security - simplyblock, accessed on January 17, 2026,
<https://www.simplyblock.io/blog/underated-postgres-multi-tenancy-with-row-level-security/>
 24. Multi-Tenant Databases with Postgres Row-Level Security - Midnyte City, accessed on January 17, 2026,

<https://www.midnytecity.com.au/blogs/multi-tenant-databases-with-postgres-row-level-security>

25. Using Row-Level Security in Prisma | Atlas Guides, accessed on January 17, 2026, <https://atlasgo.io/guides/orms/prisma/row-level-security>
26. JWT authentication: Best practices and when to use it - LogRocket Blog, accessed on January 17, 2026, <https://blog.logrocket.com/jwt-authentication-best-practices/>
27. LocalStorage vs Cookies: the best-practice guide to storing JWT tokens securely in your front-end, accessed on January 17, 2026, <https://www.cyberchief.ai/2023/05/secure-jwt-token-storage.html>
28. Cookie-based Auth vs Header-based Auth in Fullstack Apps | by Temka B - Medium, accessed on January 17, 2026, <https://medium.com/@thxdeadshotxht/cookie-based-auth-vs-header-based-auth-in-fullstack-apps-d33f34ed8fb4>
29. Render vs Vercel - GetDeploying, accessed on January 17, 2026, <https://getdeploying.com/render-vs-vercel>
30. Top 7 Free Deployment Services for 2025 (Compared with Pricing & Features) - Kuberns, accessed on January 17, 2026, <https://blogs.kuberns.com/post/top-7-free-deployment-services-for-2025-compared-with-pricing--features/>
31. Best Free Backend Hosting Platforms in 2025 — Render, Railway, Heroku, Netlify Functions & Vercel - Shreyas Kallurkar, accessed on January 17, 2026, <https://shreyaskallurkar.com/blog/best-free-hosting-platforms-for-backend-projects-2025>